

“

Bocconi University

Department of Computing Sciences

Mathematical Modelling for Neuroscience - Notes

Course Code: 30563

Sofia Lambro

Mathematical and Computing Sciences for Artificial Intelligence

Academic Year 2025/2026

“

Contents

1	Neural Encoding	2
1.1	Neural Encoding I: Firing Rates and Spike Statistics	2
1.2	Neural Encoding II: Reverse Correlation and Receptive Fields	6
2	Neural Decoding	9
2.1	Neural decoding: reading populations of neurons	9
2.2	Information theory and efficient coding	12
3	Neuronal biophysics	15
3.1	Membrane dynamics: an electrical system	15
3.2	Synapses: sources of input currents	18
3.3	Firing dynamics: from input currents to spike generation	21
4	Networks of neurons	26
4.1	From single neurons to network dynamics	26
4.2	Feedforward and linear recurrent networks	29

Chapter 1

Neural Encoding

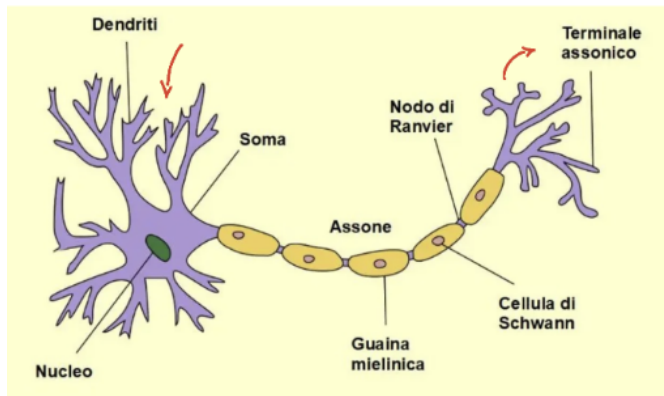
1.1 Neural Encoding I: Firing Rates and Spike Statistics

The central problem of neuroscience is to understand how the brain represents information. Neurons communicate through sequences of electrical events called action potentials or spikes, and these spikes propagate reliably over long distances and therefore constitute the fundamental units of neural communication. The key question is therefore: **how does a sequence of spikes encode information about the external world or internal states?**

The relationship between stimuli and neural responses can be studied from two complementary perspectives: **neural encoding** describes the mapping from stimulus to response, while **neural decoding** attempts to reconstruct the stimulus from the observed neural activity. In this chapter, we focus on encoding, that is, how stimuli are transformed into spike trains.

Basic Biophysics of Neurons:

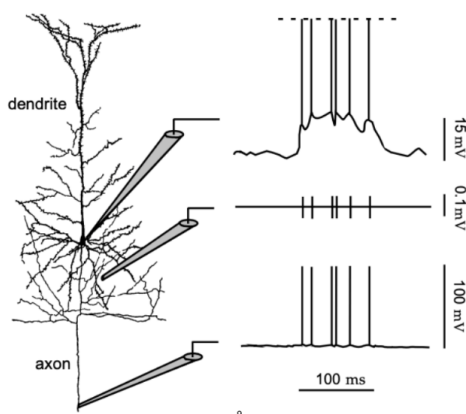
- Neuron: A specialized cell in the nervous system responsible for transmitting and processing information.
- Soma (Cell Body): The main body of the neuron containing the nucleus and most of the organelles, where essential metabolic processes occur.
- Dendrites: Branch-like extensions from the soma that receive incoming signals from other neurons or sensory receptors, information IN.
- Axon: A long, slender extension that transmits electrical signals away from the cell body to other neurons, muscles, or glands, information OUT.
- Synapses: Zone where the axon of one neuron sends information to another neuron thru its dendrites, the signal is transmitted by electrical/chemical processes as axion and dendrides do not physically touch (small space between them, approximately 20nm).



The electrical state of a neuron is described by its membrane potential, defined as the difference between intra-cellular and extracellular voltage. At rest, this potential is approximately -70 mV, maintained by ionic concentration gradients and membrane channels. Ions such as Na^+ , K^+ , Ca^{2+} , and Cl^- flow across the membrane through channels whose opening depends on voltage and biochemical signals.

When the membrane potential becomes less negative (depolarization) and crosses a threshold, a nonlinear positive feedback mechanism triggers an **action potential**, i.e. a rapid electrical/chemical signal in the communication thru synapses, which involves depolarization, repolarization and hyperpolarization phases. After a spike, the neuron enters a refractory period, during which it is difficult or impossible to fire again. This introduces a fundamental temporal structure in spike trains. Spikes propagate along the axon without attenuation, and that's why they are the primary carriers of long-range information.

Synaptic Transmission: when a spike (i.e. an action potential) occurs in the presynaptic neuron, some voltage-gated calcium channels open in the axon terminals. This allows calcium ions to influx, which triggers the release of neurotransmitters. These neurotransmitters influence the postsynaptic neuron by opening ion channels that either depolarize (excitation) or hyperpolarize (inhibition) the cell. Thus, neurons interact through a combination of electrical and chemical processes.



How do we characterize neural responses? A fundamental difficulty arises because neural responses are highly variable: even when the same stimulus is presented repeatedly, the exact spike times (the raster plot) differ across trials. This variability arises from intrinsic noise and is due to the complex and variable neuronal behaviours, therefore, instead of deterministic descriptions, we adopt a probabilistic framework (typically Poisson process with average firing rate time dependent).

The main issues are:

- Temporal Dynamics: neural responses evolve over time, influenced by both stimulus dynamics and intrinsic neural dynamics.
- Noise and Variability: neural responses exhibit inherent noise, with spike times seemingly fluctuating randomly and varying from trial to trial.

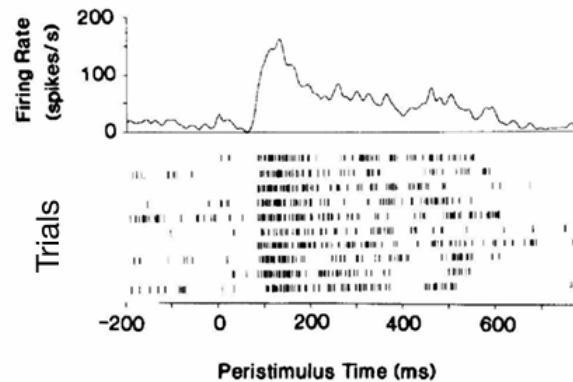
Spike Trains and the Neural Response Function: We start from computing average neural activity (firing rate). The idea is purely empirical and discrete: we repeat presentation of a stimulus n_T times, that is, over many trials, and then quantify the aver-

age response by the **post-stimulus time histogram** (PSTH). The procedure is exactly the one shown in the slides: choose a bin size Δt , count the total number of spikes n_s in each bin over all trials, and define

$$\text{PSTH} = \frac{n_s}{n_T \Delta t}.$$

This is the **instantaneous firing rate**.

So here the firing rate is introduced in the most direct way: it is the average number of spikes per trial, per unit time, inside a small time bin. We are not yet building a probabilistic model, we are simply averaging the neural activity across repetitions of the same stimulus. Before a stimulus is shown, neurons usually have low firing rates, with average rate around 1–10/s, and this baseline activity is highly variable from neuron to neuron. When a stimulus is shown, some neurons increase their rates, others decrease their rates, but the average rate across neurons increases.



The main feature emphasized in the slides is that encoding is often sparse: only a small fraction of neurons responds to a given stimulus, and vice versa a given neuron responds only to a small fraction of all possible stimuli. This is exactly what is meant by **sparse coding**.

After this, the slides introduce **characterizing neural selectivity with spike-triggered average (STA)**. Response tuning curves characterize the average neuron response to stimuli, STA is the complementary procedure: instead of asking *what is the average response given a specific stimulus*, we ask the opposite question, namely *given that a neuron fired, what was the stimulus just before?* The slides emphasize exactly this reversal of viewpoint: we measure neural responses to different stimuli and ask which stimuli trigger a response. In these slides, the response used as trigger is a **spike**, although the same approach can be generalized to multiple spikes or to firing rates. The focus here is on **time-varying stimuli**, so the question becomes: what did the stimulus do before the spike, on average?

Mathematically (see p.7), the slides define a time-varying stimulus $s(t)$, and the measured spike train is written as

$$\rho(t) = \sum_{i=1}^n \delta(t - t_i).$$

Then the **spike-triggered average** is computed as

$$C(\tau) = \frac{1}{n} \sum_{i=1}^n s(t_i - \tau).$$

This formula should be read very directly: for each spike time t_i , we look at the value of

the stimulus a time τ before the spike, namely $s(t_i - \tau)$, and then average over all spikes. So $C(\tau)$ is the mean stimulus preceding a spike by a delay τ . That is the whole intuition of STA, and at this stage that is exactly what we need.

We now build the **relationship between STA and the correlation between the stimulus and the firing rate**. The result written is that

$$\text{STA } C(\tau) \approx \frac{1}{\langle n \rangle} \left\langle \sum_{i=1}^n s(t_i - \tau) \right\rangle$$

and then, using the spike-train representation,

$$\sum_{i=1}^n s(t_i - \tau) = \int dt \rho(t) s(t - \tau).$$

Averaging over trials replaces $\rho(t)$ by the firing rate $r(t)$, so one gets

$$C(\tau) = \frac{1}{\langle n \rangle} \int dt r(t) s(t - \tau).$$

The slide then introduces the correlation between stimulus and firing rate as

$$Q_{rs}(\tau) = \frac{1}{T} \int_0^T dt r(t) s(t + \tau),$$

and concludes that (full derivation p.17)

$$C(\tau) = \frac{T}{\langle n \rangle} Q_{rs}(-\tau).$$

So the message is simple: **STA is proportional to the correlation between the stimulus and the firing rate**, evaluated at negative lag, because we are looking backward in time from the spike to the stimulus that preceded it.

Up to now we have discussed average responses, but neural responses are noisy, so the question is whether we can build simple models that capture the statistics of these fluctuations. The slides distinguish two types of noise: **interspike interval variability** i.e how much the time between two consecutive spikes changes and **trial-to-trial variability** i.e how much the spike trial changes each time we measure it. This is the transition in the slides: first we describe the mean, then we ask how responses fluctuate around the mean.

Interspike interval variability: spike trains are often characterized through the **interspike interval** (ISI), that is, the time between two consecutive spikes. ISI statistics are well described by a Poisson process and they have $CV_{ISI} = 1$, which is the coefficient of variation defined as $CV_{ISI} := \frac{\sigma}{\mu}$, that is, standard deviation divided by mean of the interspike intervals $\{t_{i+1} - t_i\}$.

The interpretation given in the slides is very important: a coefficient of variation equal to one is exactly what we expect from a continuous homogeneous Poisson process.

Trial-to-trial variability: we quantify the trial-to-trial variability noise through the **Fano Factor** (FF). The procedure is again very direct: measure responses to a single stimulus multiple times, choose a bin size T , compute mean and variance of the spike count SC in each bin over trials, and then define

$$FF = \frac{\text{Variance}(SC)}{\text{Mean}(SC)} = \frac{\sigma^2}{\mu}.$$

For a Poisson process, the slides state that $FF = 1$, so, just like CV_{ISI} compares temporal irregularity to the Poisson case, the Fano factor compares trial-to-trial count variability to the Poisson case.

Finally, typical features of Fano Factors are that before a stimulus is shown, Fano factors are often high, typically > 1 . When a stimulus is shown, Fano factors decrease, but typically stay ≥ 1 . The interpretation suggested is that values $FF > 1$ can be captured by an inhomogeneous Poisson process, where the firing rate varies in time, or more generally by a doubly stochastic process, that is, a Poisson process with stochastic rate.

1.2 Neural Encoding II: Reverse Correlation and Receptive Fields

We start from the general goal: **estimating firing rate with reverse-correlation methods**. The problem is the following: we observe a stimulus $s(t)$ and an observed neural response $r(t)$, and we want to infer the function $r_{\text{est}}(t) = f(s)$ that predicts the firing rate from the stimulus. The problem is that a simple dependence on the instantaneous stimulus (like a fine-tuned curve) is not enough, because neurons are influenced by what happened in the past, therefore the estimate must be history dependent.

This leads to writing the firing rate estimate as

$$r_{\text{est}}(t) = r_0 + \int_0^\infty D(\tau) s(t - \tau) d\tau,$$

which means that the firing rate at time t depends on a **weighted average of past stimulus values**, where each delay τ contributes with weight $D(\tau)$ called the kernel. The more general description comes from the Volterra expansion, but in practice we focus on the first-order (linear) term.

There are different approaches from this formula, one goal is to infer the **optimal kernel** $D(\tau)$ from data: this can be done by defining an error between predicted and observed firing rate and minimizing it.

In the important case where the stimulus is **white noise**, the optimal linear kernel that predicts the firing rate is found to be proportional to the spike-triggered average, more precisely:

$$D(\tau) = \frac{\langle r \rangle}{\sigma_s^2} \cdot C(\tau)$$

Another goal is finding the **most effective stimulus**: given the model for $r_{\text{est}}(t)$, one can ask which stimulus maximizes the response. The result is that the most effective stimulus is proportional to the kernel $D(\tau)$ (controlla che questa intuizione sia corretta),

so the kernel can be interpreted as the stimulus pattern that most strongly drives the neuron.

Rate estimates with static nonlinearities: when comparing predicted and measured firing rates (for example in responses to moving stimuli), one observes that the linear estimate works well when the firing rate varies slowly, but it fails to capture high-frequency and high-amplitude fluctuations, which indicates that purely linear models are not sufficient.

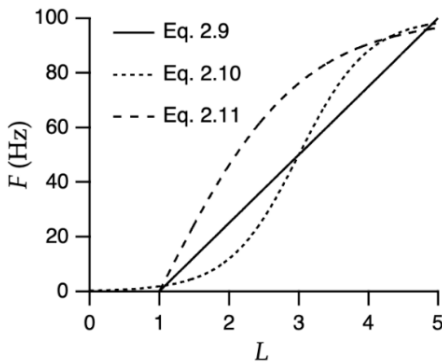
With the optimal kernel, the firing rate estimate is a linear function of the stimulus,

$$r(t) = r_0 + L(t), \quad L(t) = \int_0^\infty D(\tau)s(t - \tau)d\tau,$$

but this has several limitations: it can predict negative firing rates and has no upper bound. Therefore we introduce nonlinear functions, known as **static nonlinearities**,

$$r(t) = r_0 + F(L(t)).$$

Typical choices for F are inspired by biophysics: for example a rectified linear function



$$F(L) = G[L - L_0]_+,$$

or a sigmoidal function

$$F(L) = \frac{r_{\max}}{1 + \exp[g_1(L_{1/2} - L)]}.$$

The idea is that the neuron first performs a linear filtering of the stimulus, and then transforms this result through a static nonlinearity.

The nonlinearity can also be inferred from data: first build a linear estimate $L(t)$ using the optimal kernel, then generate a scatter plot of $r(t)$ versus $L(t)$, and finally fit a function F to this relationship. The nonlinearity is called static because it depends only on the instantaneous value $L(t)$, not on its history.

Moreover, to model spike trains rather than just firing rates, we introduce the **Linear-Nonlinear-Poisson (LNP) model**: the idea is simple, use the rate estimate $r(t)$ to drive a Poisson spike generator. So the model consists of three stages: linear filter, static nonlinearity, and stochastic spike generation.

All of these ideas have an important practical application in the **early visual system**. In this case, the stimulus is not a scalar function $s(t)$, but a function of space and time $s(x, y, t)$, representing the contrast at each pixel and time.

The reverse-correlation framework extends naturally and for images we define

$$C(x, y, \tau) = \frac{1}{\langle n \rangle} \sum_i s(x, y, t_i - \tau),$$

with rate estimate

$$L(t) = \int_0^\infty d\tau \iint dx dy D(x, y, \tau) s(x, y, t - \tau),$$

and with kernel which depends on both space and time

$$D(x, y, \tau) = \frac{\langle r \rangle C(x, y, \tau)}{\sigma_s^2}.$$

This leads to the concept of **spatio-temporal receptive fields**. The receptive field is defined as the set of points (x, y) for which $D(x, y, \tau) \neq 0$. There are two important cases:

separable: $D(x, y, \tau) = D_s(x, y)D_t(\tau)$, non-separable: $D(x, y, \tau) \neq D_s(x, y)D_t(\tau)$.

In the **thalamus and retina**, receptive fields have a characteristic structure: a circular central ON (or OFF) region surrounded by an annular OFF (or ON) region, fitted by a difference of Gaussians. While in **primary visual cortex (V1)**, the spatial receptive field of a simple cell is instead fitted by a Gabor function. Finally, receptive fields also have a temporal structure, meaning that their response evolves over time, typically showing a sequence of positive and negative phases. This temporal component, combined with the spatial structure, determines how neurons respond to dynamic visual stimuli.

Chapter 2

Neural Decoding

2.1 Neural decoding: reading populations of neurons

We now move from encoding to decoding, that is, from describing how neurons respond to a stimulus to understanding how the stimulus can be reconstructed from neural activity. From our point of view as neuroscientists, decoding provides a tool to understand what information is actually present in neural activity and how it is organized across populations of neurons.

The problem is set up as follows: a stimulus is described by a parameter θ , for example the orientation of a bar or the direction of movement; in response to this stimulus, we observe the activity of N neurons over a given time window, and we summarize this activity as a vector of responses

$$\vec{r} = (r_1, \dots, r_N),$$

for instance spike counts. The goal is to construct a function $\hat{\theta}(\vec{r})$, called a decoder, that estimates the stimulus from the neural responses. This is inherently difficult because of noise: even when the stimulus is fixed, neural responses vary from trial to trial. Therefore, decoding is fundamentally a statistical problem.

Example 1: population average

To make this more concrete, we start from simple examples of decoders. The first one is the population average: suppose that each neuron responds linearly to the stimulus, with some noise,

$$r_i = \theta + z_i, \quad z_i \sim \mathcal{N}(0, 1).$$

A very natural estimator is simply the average across neurons,

$$\hat{\theta}(\vec{r}) = \frac{1}{N} \sum_i r_i.$$

The intuition is straightforward: each neuron provides a noisy measurement of the same underlying quantity θ , and averaging reduces the noise, therefore increasing N improves

accuracy.

$$r_i = \theta + z_i, \quad z_i \sim \mathcal{N}(0, 1) \quad \hat{\theta}(\vec{r}) = \frac{1}{N} \sum_i r_i$$

Example 2: population vector

However, not all neural codes are so simple as in many cases individual neurons are not sufficient to represent the full stimulus space. For example, in systems encoding direction, each neuron is tuned to a preferred direction, and multiple neurons respond to the same stimulus. This leads to the idea of a population vector decoder: each neuron i is associated with a preferred direction θ_i , and we represent it as a unit vector $\vec{c}_i$. The response of the neuron depends on how close the stimulus is to its preferred direction, for example through a cosine tuning curve. The decoding strategy is then to compute a weighted average of these preferred directions,

$$\hat{\vec{C}} = \frac{1}{N} \sum_i r_i \vec{c}_i.$$

In this way, each neuron “votes” for its preferred direction, with strength proportional to its activity, and the final estimate is given by the direction of the resulting vector. The key intuition is that information is distributed across the population: no single neuron is sufficient, but together they form a coherent representation.

$$r_i = r_{\max} \cos(\theta - \theta_i), \quad \vec{c}_i = (\cos \theta_i, \sin \theta_i), \quad \hat{\vec{C}} = \frac{1}{N} \sum_i r_i \vec{c}_i$$

Quantify the error of the estimator: having defined decoders, we now need a way to evaluate how good they are. Given an estimate $\hat{\theta}(\vec{r})$, the natural question is how far it is from the true stimulus θ , which leads to defining the error of the estimator.

The first quantity is the **bias**,

$$b_{\hat{\theta}} = \langle \hat{\theta}(\vec{r}) \rangle - \theta,$$

which measures systematic deviations from the true value. An estimator is called unbiased if this quantity is zero.

The second quantity is the **mean squared error (MSE)**,

$$\langle (\hat{\theta}(\vec{r}) - \theta)^2 \rangle,$$

which measures the typical magnitude of the error. A good decoder should ideally be unbiased and have a small MSE.

At this point, a deeper question arises: is there a fundamental limit to how well we can decode? This is where the concept of **Fisher information** enters. To define it, we introduce the score, which measures how sensitive the likelihood of the data is to changes in the stimulus,

$$V(\theta, \vec{r}) = \frac{\partial}{\partial \theta} \log P(\vec{r} | \theta).$$

The Fisher information is then defined as the variance of this quantity,

$$J(\theta) = \langle V(\theta, \vec{r})^2 \rangle.$$

Intuitively, Fisher information quantifies how much information the neural response carries about the stimulus: if small changes in θ strongly affect the likelihood, then the stimulus can be estimated more precisely.

The central result connecting decoding performance and Fisher information is the **Cramér-Rao inequality**, which states that for any unbiased estimator,

$$\langle (\hat{\theta}(\vec{r}) - \theta)^2 \rangle \geq \frac{1}{J(\theta)}.$$

This provides a lower bound on the error: no unbiased decoder can do better than this. The best possible decoders are those that achieve this bound, and they are called **efficient estimators**. In the simple example of linear neurons with Gaussian noise, the population average estimator turns out to be efficient, meaning that averaging is not just intuitive, but actually optimal.

A general and very important decoding strategy is the **maximum likelihood estimator (MLE)**, defined as

$$\hat{\theta}_{ML} = \arg \max_{\theta} P(\vec{r}|\theta).$$

The idea is to choose the stimulus value that makes the observed neural response most likely. In large populations, the MLE becomes approximately Gaussian distributed and achieves the Cramér-Rao bound, so it is efficient. In the simple linear Gaussian example, the MLE coincides with the population average, showing again that averaging is optimal in that case.

Finally, we consider a more realistic model of neural responses, where neurons fire as independent Poisson processes with tuning curves $r_i(\theta)$. In this case, the variability of responses depends on the firing rate itself, and the Fisher information becomes

$$J(\theta) = T \sum_i \frac{r'_i(\theta)^2}{r_i(\theta)}.$$

This expression provides important intuition: information increases with observation time T , and it is large where the tuning curves are steep, that is, where small changes in the stimulus produce large changes in firing rate.

If we consider a population of neurons with similar tuning curves but different preferred stimuli uniformly distributed, then in the large population limit the Fisher information scales as

$$J \sim NT,$$

and therefore the decoding error scales as

$$\text{error} \sim \frac{1}{NT}.$$

This highlights a fundamental principle of population coding: accuracy improves with both the number of neurons and the duration of observation, showing how reliable representations can emerge from noisy individual components.

2.2 Information theory and efficient coding

So far, decoding has been treated as a problem of statistical estimation: given a neural response, we construct a decoder and evaluate how well it reconstructs the stimulus, for example through bias, mean squared error, and bounds such as the Cramér-Rao inequality. This approach focuses on specific estimators and their performance. We now take a step back and ask a more fundamental question: independently of any particular decoder, how much information about the stimulus is actually contained in the neural response?

This is exactly why information theory is introduced: instead of asking how well a given decoder performs, we want to quantify the intrinsic relationship between stimulus and response. The stimulus is described by a probability distribution $P(\theta)$, while the neural response is described by the conditional distribution $P(r|\theta)$. The question becomes: how much does observing r reduce our uncertainty about θ ? This shift is important, because it separates what is in principle decodable from the particular method we use to decode it.

To formalize this idea, we introduce the concept of entropy: for a random variable X , entropy is defined as

$$H(X) = - \sum_i p_i \log_2 p_i,$$

and it measures the uncertainty associated with X . If one outcome is certain, entropy is zero, if all outcomes are equally likely, entropy is maximal, and in this sense, entropy provides a quantitative notion of unpredictability. When we consider two variables, such as stimulus and response, we can define joint entropy $H(X, Y)$ and conditional entropy $H(X|Y)$, which measures how much uncertainty remains about X after observing Y . The key quantity for decoding is the mutual information

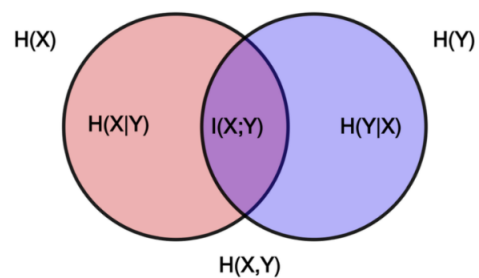
$$I(X; Y) = \sum_{x,y} p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)},$$

which measures how much uncertainty about X is reduced by observing Y . An equivalent and very intuitive expression is

$$I(X; Y) = H(X) - H(X|Y).$$

So mutual information is exactly the reduction in uncertainty about the stimulus once we observe the neural response. If stimulus and response are independent, observing the response tells us nothing and the information is zero; if the response perfectly determines the stimulus, the uncertainty is completely removed and the information is maximal.

At this point, the connection with decoding becomes clear: mutual information tells us how well any decoder could possibly perform, regardless of its specific form. It is therefore

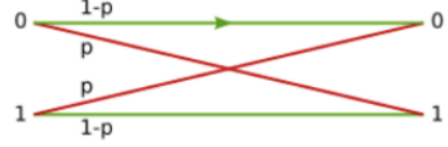


a fundamental limit, complementary to the Cramér-Rao bound.

Example 1: noisy binary symmetric channel

To build intuition, it is useful to consider a few simple examples. In a binary channel, where the stimulus can take two values with equal probability and noise flips the output with probability p , the mutual information is

$$I = 1 - H_b(p)$$



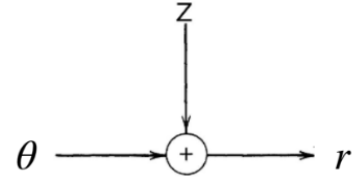
where $H_b(p) = -p \log_2(p) - (1-p) \log_2(1-p)$ is the binary entropy.

When there is no noise, the output perfectly reflects the input and information is maximal, while when noise is maximal, the output becomes independent of the input and information vanishes. The role of noise is therefore directly visible as a reduction of information.

Example 2: noisy Gaussian channel

In a Gaussian channel, where the response is given by

$$r = \theta + z,$$



with Gaussian signal and noise, the mutual information becomes

$$I = \frac{1}{2} \log_2 \left(1 + \frac{\sigma_\theta^2}{\sigma_N^2} \right).$$

Here the key quantity is the signal-to-noise ratio: stronger signals or lower noise increase the amount of information transmitted. This example connects directly with the intuition developed earlier for decoding accuracy.

Example 3: Poisson neuron tuned to a 1D stimulus

In a more realistic neural setting, we consider a Poisson neuron with tuning curve $\phi(\theta)$. The probability of observing n spikes given θ in an interval of duration t is Poisson distributed and depends on the firing rate, so the mutual information takes the form

$$I \approx t \int d\theta P(\theta) \phi(\theta) \log_2 \frac{\phi(\theta)}{\langle \phi \rangle}.$$

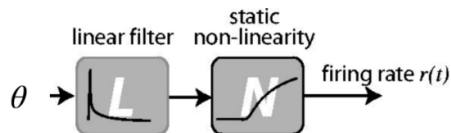
This shows that the structure of the tuning curve itself determines how much information is transmitted: not all response functions are equally informative.

Having introduced information as a measure of coding performance, we can now ask a new type of question, which was not present before: *are neural systems organized in a way that maximizes information transmission?* This is the idea behind the efficient coding hypothesis, originally proposed by Attneave and Barlow. Instead of simply describing neural responses, we now try to explain them as the result of an optimization principle.

To make this concrete, consider again a neuron described by a linear stage followed by a static non-linearity,

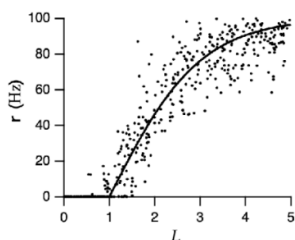
$$r = \phi(\theta) + z.$$

Previously, this non-linearity was introduced as a phenomenological component to better fit neural responses. Now it takes on a different role: it becomes an object we can optimize. The goal is to choose ϕ so as to maximize the mutual information between stimulus θ and response r .



Using the expression

$$I = H(r) - H(r|\theta),$$



and observing that for additive Gaussian noise the conditional entropy $H(r|\theta)$ is fixed, maximizing information reduces to maximizing the entropy of the response $H(r)$. This leads to a simple but powerful result: the output distribution should be as spread out as possible.

Under bounded output, the entropy is maximized when the output distribution is uniform. This implies that the non-linearity ϕ should transform the stimulus in such a way that the response is uniformly distributed. In practice, this corresponds to performing histogram equalization: the non-linearity should be proportional to the cumulative distribution function of the stimulus.

The intuition is very clear: if some output values are used more often than others, part of the dynamic range is wasted, and by redistributing responses so that all output levels are equally likely, the neuron uses its full range efficiently and maximizes the information it transmits. This provides a deep reinterpretation of the static non-linearity: it is not only a descriptive feature of neural responses, but a functional adaptation that optimizes information transmission. In this way, the introduction of information theory completes the picture, connecting encoding, decoding, and the underlying principles that may govern neural representations.

Chapter 3

Neuronal biophysics

3.1 Membrane dynamics: an electrical system

Up to now, the focus of the course has been on describing neural activity and understanding how information is encoded and decoded: we have built models that tell us how a stimulus is transformed into a response and how much information that response carries. In doing so, we have mostly treated the neuron as an abstract input-output system.

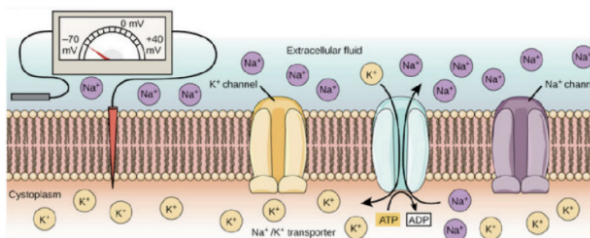
At this point, however, the perspective changes: we are no longer satisfied with describing what the neuron does, instead, we want to understand why and how it does it.

In particular, we now ask a fundamentally different question: why does a signal like $r(t)$ arise at all, and what physical mechanisms produce it? This is the entry point to mechanistic models.

This shift is best understood by recalling the key experimental observations: in the absence of inputs, neurons sit at a membrane potential of about -70 mV, meaning that the inside of the cell is negatively charged relative to the outside.

When a small current is injected, the membrane potential does not jump abruptly, but rather relaxes smoothly toward a new equilibrium value, following an exponential time course with a characteristic timescale of about 10–20 ms, and only when the current becomes large enough do action potentials appear. The goal of what follows is to explain the first two behaviors, and in particular to understand both the exponential relaxation and the origin of the resting potential.

Neuronal membrane: to begin, we look at the structure of the neuronal membrane. The membrane is a lipid bilayer that isolates/separates the inside from the outside of the cell. Because lipids are poor conductors of ions, the membrane effectively blocks charge flow, and this allows charges to accumulate on its two sides. This observation immediately suggests a powerful physical analogy: the membrane behaves like a capacitor!



The membrane potential is defined as $V = V_{in} - V_{out}$, and by convention the outside

is taken to be at zero potential. Since $V_{\text{out}} = 0$, the fact that $V \approx -70 \text{ mV}$ implies that V_{in} must be negative. Moreover, the capacitor intuition is very explicitly: when negative charges accumulate on one side and positive charges on the other, separated by an insulating membrane, we are precisely in the situation of a capacitor.

Neuron as a capacitor: IF we momentarily ignore ionic channels and pumps, this capacitor picture becomes exact and sufficient to correctly describe the behaviour: injecting a current into the neuron simply leads to charge accumulation on the membrane, with negative charge building up on the inside and positive charge on the outside.

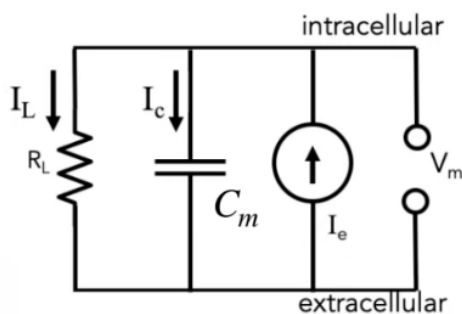
The amount of stored charge is related to the voltage through $Q = C_m V_m$, and since current is the rate of change of charge $I = dQ/dt$, we obtain the relation,

$$I_C = C_m \frac{dV_m}{dt}.$$

The meaning of this expression in intuitive terms is that this is the “fake” current generated inside a capacitor when the voltage changes in time. Even though charges do not cross the membrane, changing the voltage requires moving charge onto the membrane surfaces, and this is what appears as a current.

Kirchhoff’s current law then tells us that the injected current must equal the capacitive current, so that $I_e = C_m dV_m/dt$. This explains why voltage does not change instantaneously, but instead evolves over time.

Neuron as a leaky capacitor: however, this picture is still incomplete as in reality, the membrane is not a perfect insulator: ions can cross it through channels, and therefore some of the injected current does not go into charging the capacitor but instead leaks across the membrane. For this reason, we must introduce a resistor in parallel with the capacitor, which leads to the second approximation: the neuron behaves as a leaky capacitor.



In this model, the leakage of ions is described by a resistance R_L , and the corresponding current obeys Ohm’s law, $I_L = V_m/R_L$. The current balance now becomes $I_e = I_C + I_L$, meaning that part of the current charges the membrane while part leaks away. Substituting the expressions for the two currents gives

$$I_e = C_m \frac{dV_m}{dt} + \frac{V_m}{R_L}.$$

At this point, the derivation simply proceeds by rearranging the terms step by step. First we isolate the derivative,

$$C_m \frac{dV_m}{dt} = I_e - \frac{V_m}{R_L},$$

and then divide by C_m ,

$$\frac{dV_m}{dt} = \frac{I_e}{C_m} - \frac{V_m}{R_L C_m}.$$

Introducing the membrane time constant $\tau_m = R_L C_m$, this becomes

$$\frac{dV_m}{dt} = -\frac{V_m}{\tau_m} + \frac{I_e}{C_m}.$$

This as a first-order linear differential equation, and its interpretation is simple but crucial: the neuron integrates the input current, but at the same time the voltage leaks back toward equilibrium. The capacitor stores charge, while the resistor continuously dissipates it, and the balance between these two effects determines the dynamics.

If we now inject a step current (i.e a controlled current, the external current in the derivation above) $I_e(t) = I_0 \Theta(t)$, the solution of this first-order linear differential equation is an exponential relaxation,

$$V(t) = V_\infty + (V_0 - V_\infty)e^{-t/\tau_m}, \quad V_\infty = I_0 R_L.$$

This matches exactly what is observed experimentally: the membrane potential approaches a new steady value exponentially, with a time constant τ_m . The leaky capacitor model therefore provides a mechanistic explanation of the temporal response of neurons to small inputs, we have now understood why the response has an exponential behavior (10-20 ms).

Nernst and Goldman equation: at this point, however, one important question remains. Why does the membrane settle around -70 mV when there is no input? To answer this, we must move beyond the circuit analogy and consider the movement of ions across the membrane.

When an ion channel is open, ions are subject to two competing forces:

1. Diffusion: if there is a concentration gradient, ions tend to move from regions of high concentration to regions of low concentration.

This is described by Fick's law, $J_{\text{diff}} = -D \Delta[C]$.

2. Electric force: as ions move, they create a charge imbalance that generates an electric field, which in turn pushes them in the opposite direction.

This is described by a drift term, $J_{\text{drift}} = -\mu z[C] \Delta V$.

The slides illustrate this with potassium: because potassium concentration is higher inside the cell, it tends to diffuse outward, but the resulting negative charge inside pulls it back in. At equilibrium, these two fluxes cancel, $J_{\text{diff}} + J_{\text{drift}} = 0$, and this condition leads to the **Nernst equation**,

$$V_{eq} = -\frac{V_T}{z} \log \frac{[C]_{\text{in}}}{[C]_{\text{out}}}.$$

This equation gives the equilibrium, or reversal, potential for a single ion species: by plugging in the concentrations of an ion, we obtain the voltage at which diffusion and

electrical forces balance; however, this equation applies to one ion at a time, whereas the membrane is permeable to multiple ions simultaneously (D , μ , z , V_T are just constants, do not care deeply about them, what matters is the ratio of the concentrations).

To account for this, we use the **Goldman equation**, which combines the contributions of different ions:

$$V_{eq} = \frac{RT}{F} \log \frac{P_K[K^+]_{out} + P_{Na}[Na^+]_{out} + P_{Cl}[Cl^-]_{in}}{P_K[K^+]_{in} + P_{Na}[Na^+]_{in} + P_{Cl}[Cl^-]_{out}}.$$

Here the permeabilities P_i weight the influence of each ion. This equation predicts a resting potential of approximately -70 mV, in agreement with experimental observations.

The intuition is that each ion “pulls” the membrane toward its own equilibrium value, and the actual resting potential is the result of a balance between these competing effects.

Neuron as a leaky capacitor with ionic channels: finally, we can combine all these elements into a unified picture. The membrane behaves as a capacitor, storing charge. Ionic channels provide conductive pathways, modeled as resistances or conductances. Each ionic species has a reversal potential, determined by the balance of diffusion and drift. The membrane potential evolves according to the balance between capacitive current, ionic currents, and external input.

This leads to the general equation

$$C_m \frac{dV}{dt} = - \sum_i g_i (V - E_i) + I_e,$$

where each term $g_i(V - E_i)$ represents the current through a specific ionic channel type. In this way, the neuron emerges as an electrical circuit whose behavior is entirely determined by physical principles. The exponential relaxation is explained by the leaky capacitor structure, and the resting potential is explained by ionic equilibrium.

The sum is over the different channels types, more specifically in the **Hodgkin-Huxley** model the contributions will be from Na^+ channels pushing voltage up (spike initiation), K^+ channels bringing voltage down (recovery), and Leak channels continuously stabilizing the system. Together, these mechanisms account for the basic electrophysiological properties observed in neurons.

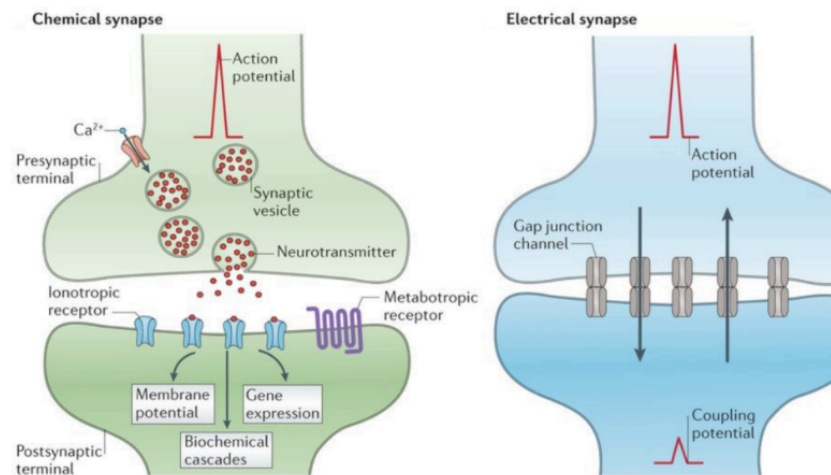
3.2 Synapses: sources of input currents

Having understood how a single neuron behaves as an electrical circuit, we now naturally move to the next step: neurons in the brain do not operate in isolation, but interact with each other. In the previous part, currents were injected artificially by an experimenter using electrodes, and we studied how the membrane responds to such inputs. The key idea now is that in the brain, these currents are not externally imposed, but are generated by other neurons. Synapses are precisely the mechanism through which this interaction takes place.

A synapse is a specialized junction between two neurons, typically connecting the axon of a presynaptic neuron to the dendrite of a postsynaptic neuron. The small protrusions visible

on neuronal structures, such as dendritic spines and axonal boutons, are the physical sites where synapses are formed. The important point is that synapses generate currents in the postsynaptic neuron, and therefore they act exactly like the external currents we previously studied. This creates a very important conceptual bridge: all the mechanisms we developed for membrane potential dynamics under injected currents remain valid when the current is synaptic in origin.

To understand synapses, we first look at how synaptic transmission occurs. When an action potential arrives at the presynaptic terminal, it triggers the opening of voltage-dependent calcium channels. The resulting influx of calcium increases its concentration inside the terminal, which in turn



causes vesicles to fuse with the membrane and release neurotransmitter molecules into the synaptic cleft. These molecules diffuse across the cleft and bind to receptors on the postsynaptic neuron. This binding ultimately leads to the opening of ion channels and therefore to a change in the conductance of the postsynaptic membrane. The entire process can be summarized as a transformation from an electrical signal in the presynaptic neuron to a chemical signal in the synaptic cleft, and back to an electrical signal in the postsynaptic neuron.

Two types of ion channel activation. At a qualitative level, synaptic transmission can be described in terms of two main types of mechanisms for activating ion channels:

1. **Ionotropic transmission:** the neurotransmitter binds directly to an ion channel, causing it to open almost immediately. This leads to fast changes in membrane potential and allows rapid communication between neurons.
2. **Metabotropic transmission:** the neurotransmitter binds to a receptor that activates intracellular signaling pathways. These pathways then indirectly affect ion channels. This process is slower, but produces longer-lasting and more complex effects, often related to modulation and plasticity.

Two types of synapses. More broadly, synapses themselves can be divided into two main classes:

1. **Chemical synapses**, which are the dominant type in the brain, operate through neurotransmitter release as described above. They are typically slower and unidirectional, meaning that information flows from the presynaptic neuron to the postsynaptic one.

2. **Electrical synapses**, which use gap junctions that directly connect the cytoplasm of two neurons, allowing ions to flow freely between them. These synapses are very fast and bidirectional, but less flexible.

The focus in what follows will be on chemical synapses, as they provide the richness necessary for neural computation.

Mathematical description of synaptic current: we now move from this qualitative description to a mathematical one. The key observation is that synaptic transmission modifies the conductance of the postsynaptic membrane. Therefore, the synaptic current can be written in the same form as the ionic currents we introduced earlier:

$$I_s = g_s(V - E_s).$$

Here g_s is the synaptic conductance and E_s is the reversal potential associated with the synapse. This expression is completely analogous to the currents $g_i(V - E_i)$ we previously used for ionic channels. The intuition is that the synapse effectively adds a new conductance pathway to the membrane, pulling the voltage toward its own reversal potential.

The synaptic conductance itself is not constant, but depends on the state of the synapse. It can be written as

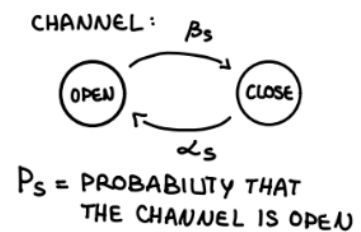
$$g_s = \bar{g}_s P = \bar{g}_s P_s P_{\text{rel}},$$

where $\bar{g}_s$ is the maximal conductance, P_{rel} is the probability that neurotransmitter is released, and P_s is the probability that postsynaptic channels are open. The conductance therefore depends both on presynaptic release and on postsynaptic channel activation, making it clear that synaptic transmission is intrinsically probabilistic.

The central quantity we now need to model is the **open probability** P_s , which describes the fraction of postsynaptic channels that are open: this probability captures the entire dynamics of the synapse such as neurotransmitter release, receptor binding, and channel opening, all summarized into this single variable.

To model P_s , we assume that channels can be either open or closed: the probability evolves according to a simple kinetic equation:

$$\frac{dP_s}{dt} = \alpha_s(1 - P_s) - \beta_s P_s.$$



This equation has a very intuitive interpretation. The first term, $\alpha_s(1 - P_s)$, represents the opening of channels, it is proportional to the fraction of channels that are currently closed, $1 - P_s$, and to an opening rate α_s , which depends on the concentration of neurotransmitter. The second term, $\beta_s P_s$, represents the closing of channels, with a constant closing rate β_s .

This model allows us to understand the time course of synaptic responses: in the absence of a presynaptic spike, the neurotransmitter concentration is effectively zero, so $\alpha_s \approx 0$, and the equation reduces to

$$\frac{dP_s}{dt} = -\beta_s P_s,$$

which means that the open probability decays exponentially to zero. During a presynaptic spike, neurotransmitter is released, α_s becomes large, and channels open rapidly. After the spike, the neurotransmitter disappears, α_s goes back to zero, and the system returns to exponential decay.

More precisely, if the neurotransmitter concentration increases for a duration T , the solution of the equation is

$$P_s(t) = \begin{cases} 1 + [P_s(0) - 1]e^{-\alpha_s t}, & 0 < t < T, \\ P_s^{\max} e^{-\beta_s(t-T)}, & t > T, \end{cases}$$

where $P_s^{\max}$ is the maximum value reached during the spike (i.e. the first expression above for $t=T$). This captures the typical shape of a postsynaptic response: a rapid rise followed by a slower decay.

Common synapse types: finally, we can interpret different types of synapses in terms of their reversal potentials. Excitatory synapses, such as glutamatergic ones, have reversal potentials around 0 mV and tend to depolarize the neuron. Inhibitory synapses, such as GABAergic ones, have reversal potentials around -80 mV and tend to hyperpolarize it. The specific receptors involved (AMPA, NMDA, GABA_A, etc.) differ in their kinetics and voltage dependence, but all follow the same general principle: they modify the membrane conductance and therefore generate currents of the form $g_s(V - E_s)$.

In this way, synapses fit seamlessly into the circuit picture developed earlier. They are not fundamentally new elements, but rather additional conductances whose dynamics are controlled by presynaptic activity. The neuron is therefore no longer just a passive circuit responding to external input, but part of a network in which currents are continuously generated by other neurons through synaptic interactions.

3.3 Firing dynamics: from input currents to spike generation

At this point, we have built a fairly complete picture of how a neuron behaves at the level of its membrane. First, we understood that the membrane acts as a leaky capacitor, integrating incoming currents while simultaneously leaking charge. Then we introduced synapses, and saw that in the brain the currents entering a neuron are not externally injected, but are generated by other neurons. These synaptic currents can be written in the same form as ionic currents, and therefore they simply add new terms to the membrane equation.

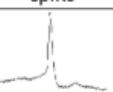
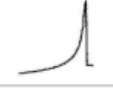
Putting everything together, the membrane potential dynamics in the presence of synaptic input becomes

$$C_m \frac{dV}{dt} = -I_m - I_s + I_e, \quad I_s = g_s(V - E_s),$$

which shows that synaptic currents enter exactly in the same way as the other currents we studied.

At this stage, however, something important is still missing: the models we have discussed so far describe how the membrane potential evolves, but they do not directly describe how

spikes are generated in a way that is simple to analyze.

Model	D	spike
HH	4	
2D reduced models (KK, ML, FN, etc)	2	
Non-linear IF models (QIF, EIF)	1	
Leaky integrate-and-fire (LIF)	1	

The Hodgkin-Huxley model provides a very accurate description of spike generation, but it is highly nonlinear and involves many variables (4 dimensional), making it difficult to study analytically or to simulate in large networks. The key idea is therefore: if we are interested in firing statistics rather than detailed biophysics, what is the minimal mechanism we need? The answer is that we can keep only the leak and the capacitance, and replace the complex spike-generating dynamics with a simple threshold mechanism.

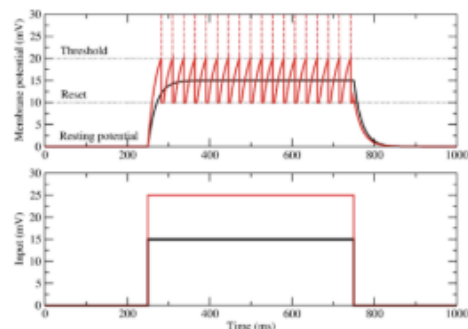
This leads to the **leaky integrate-and-fire (LIF) model**: the logic is to keep the sub-threshold membrane dynamics exactly as derived before, and to model spikes in a simplified way.

As long as the membrane potential remains below a threshold V_{th} , the dynamics is governed by

$$C_m \frac{dV}{dt} = -g_L(V - E_L) + I_{syn}(t),$$

or equivalently

$$\tau_m \frac{dV}{dt} = -(V - E_L) + \tilde{I}_{syn}(t),$$



which is the same leaky capacitor equation we have already derived.

When the voltage reaches the threshold V_{th} , a spike is emitted, and the voltage is reset to a value V_{reset} . Optionally, a refractory period τ_{rp} can be introduced, during which the neuron cannot fire. In this way, the detailed ionic mechanisms responsible for spike generation are replaced by a simple rule: threshold crossing followed by reset, V_{th} , V_{reset} and τ_r replace Na^+ and K^+ currents in the HH model.

The importance of this model lies in its simplicity as it isolates the essential ingredients needed to describe firing: integration of inputs, leakage, and a threshold mechanism, making it possible to analyze firing behavior in a mathematically tractable way.

To understand what this model predicts, it is useful to compare two different situations. If we inject a constant current into the neuron, the membrane potential increases until it reaches threshold, after which a spike is emitted and the process repeats, which produces a sequence of regularly spaced spikes. In fact, the model allows us to compute explicitly the interspike interval and therefore the firing rate as a function of the input current,

giving the so-called f - I curve:

$$r_{isi} = \frac{1}{\tau_m \log \left(\frac{R_m I_e + E_L - V_{\text{reset}}}{R_m I_e + E_L - V_{th}} \right)}$$

which captures the basic dependence of firing rate on input current and matches well the first spike response observed in experiments.

At this point, however, a discrepancy emerges: in experiments where current is injected in vitro, neurons typically fire regularly, but in vivo when neurons receive input from other neurons through synapses, their firing is highly irregular. The same neuron can switch from regular firing to seemingly random spike trains depending on the conditions. The key question therefore becomes: what mechanism generates this irregular firing?

The answer proposed in the slides is that irregular firing does not come from intrinsic neuronal properties, but from the statistics of the input current: in the cortex, neurons receive input from a very large number of presynaptic neurons, often hundreds or thousands, and about 80% of these inputs are excitatory while 20% inhibitory. Moreover each presynaptic neuron fires irregularly, often following approximately Poisson statistics with a low firing rate of about 1-10 spikes per second. The total synaptic current is therefore the sum of a large number of small, random contributions.

More precisely, we can write the total synaptic current as the sum of excitatory and inhibitory contributions,

$$I_s = I_{s,E} + I_{s,I},$$

with

$$I_{s,E} = \sum_{i=1}^N g_{E,i}(V - E_E), \quad I_{s,I} = \sum_{i=1}^N g_{I,i}(V - E_I),$$

where each term represents the current generated by a single presynaptic neuron. This makes explicit that the total input is the result of many individual synaptic events.

This observation has a very important consequence as when many independent random inputs are summed together, the resulting total input can be approximated, by the central limit theorem, as a Gaussian process. Therefore, instead of thinking of synaptic input as a sequence of discrete events, we can model it as a fluctuating current characterized by a mean and a variance:

$$\begin{aligned} \mu &= \mu_E - \mu_I + E_L, & \mu_E &= Nr_{\text{pre}}\tau_m J_E, & \mu_I &= Nr_{\text{pre}}\tau_m J_I, \\ \sigma^2 &= \sigma_E^2 + \sigma_I^2, & \sigma_E^2 &= Nr_{\text{pre}}(\tau_m J_E)^2, & \sigma_I^2 &= Nr_{\text{pre}}(\tau_m J_I)^2. \end{aligned}$$

These expressions make explicit how both the average input and its fluctuations arise from the statistics of presynaptic activity.

Under this approximation, the membrane equation becomes

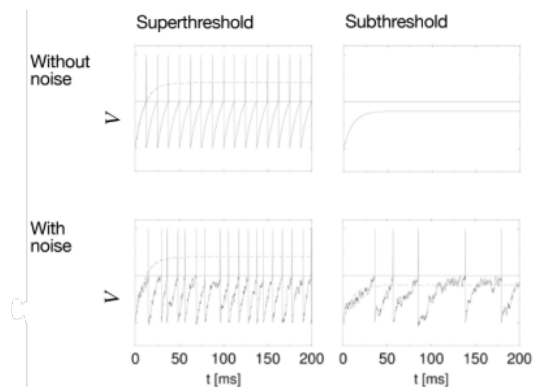
$$\tau_m \frac{dV}{dt} = -V + \mu + \tau_m \sigma z(t),$$

where μ is the mean input, σ its standard deviation, and $z(t)$ is a Gaussian white noise process with zero mean and delta correlations. This equation shows that the neuron behaves as a leaky integrator driven by a stochastic input, where the deterministic term μ sets the average drive and the noise term $\sigma z(t)$ captures fluctuations due to synaptic activity.

This leads to a fundamental insight: the firing behavior of neurons depends not only on the mean input, but also on its fluctuations.

Two distinct regimes emerge:

- In the **superthreshold regime**, the mean input μ is large enough that, even in the absence of noise, the neuron would fire regularly. Adding noise perturbs the timing of spikes but does not fundamentally change the behavior.
- In the **subthreshold regime**, the mean input is too small to reach threshold on its own, so the neuron would remain silent without noise. However, the fluctuations occasionally push the membrane potential above threshold, generating spikes in an irregular manner.



This provides a simple and powerful explanation of cortical activity: *in vitro*, where inputs are controlled and often constant, neurons fire regularly, while *in vivo*, where inputs are noisy due to synaptic interactions, neurons fire irregularly. The irregularity is therefore not a property of the neuron itself, but of the network in which it is embedded.

From a more formal point of view, the firing statistics of the neuron driven by such stochastic input can be computed, and the mean interspike interval is given by

$$t_{\text{isi}} = \tau_m \pi \int_{(V_{\text{th}} - E_L - \mu)/\sigma}^{(V_{\text{reset}} - E_L - \mu)/\sigma} \exp(u^2) (1 + \text{Erf}(u)) du, \quad r_{\text{post}} = \frac{1}{t_{\text{isi}}}.$$

Finally, the LIF model driven by noisy input not only explains qualitatively the firing patterns observed in cortex, but also quantitatively reproduces their firing statistics. The firing rate as a function of input depends strongly on the noise level: increasing the variance of the input allows the neuron to fire even when the mean input is below threshold. In this sense, noise plays a constructive role, enabling activity that would otherwise be impossible.

Big picture: from membrane to firing

A neuron can be understood as an electrical system where the membrane acts as a capacitor that stores charge and ion channels provide leak conductances, leading to leaky integration of inputs with timescale $\tau_m = R_L C_m$. The resting potential (~ -70 mV) arises from the balance of ionic fluxes (Nernst for single ions, Goldman for multiple ions). In the brain, currents are not injected externally but generated by other neurons through

synapses, which act as additional conductances $I_s = g_s(V - E_s)$. Simplifying the neuron leads to the leaky integrate-and-fire model, where voltage integrates inputs until a threshold is reached, producing spikes. The key insight is that in vivo inputs are noisy (many synapses, Poisson firing), so firing is driven by fluctuations: strong mean input produces regular firing, while noise enables irregular firing even below threshold.

Chapter 4

Networks of neurons

4.1 From single neurons to network dynamics

In the previous sections, we developed a mechanistic understanding of single neurons: we started from the membrane as a leaky capacitor, introduced synaptic inputs as currents generated by other neurons, and finally described how these inputs are transformed into spikes using the leaky integrate-and-fire model. At that point, however, an important conceptual step remains: if synaptic currents come from other neurons, then the activity of each neuron depends on the activity of many others. This means that the correct level of description is no longer the single neuron, but the *network*.

Neurons in the brain form a highly interconnected system: at the largest scale, they constitute a cohesive whole-brain network, while at smaller scales they organize into local circuits that are themselves interconnected. The activity of any given neuron is therefore determined by the collective activity of many others, and understanding neural computation requires studying these interactions.

A first important aspect of networks is their connectivity structure as there are three main types of connectivity:

1. **Feedforward** architectures: information flows in a single direction, typically from lower to higher processing stages, allowing progressive extraction of features.
2. **Recurrent** networks: neurons are connected in loops, so activity can reverberate and interact over time, enabling complex computations such as memory and integration.
3. **Feedback** connections: they propagate information from higher to lower areas, providing contextual signals and error corrections.

Another key feature of neural networks is the variability of connection strengths. Synaptic weights are not uniform: they depend on the specific properties of neurons and show strong variability across connections. This heterogeneity plays a central role in determining network dynamics.

A fundamental biological constraint is given by **Dale's law**, which states that each neuron releases the same type of neurotransmitter at all of its synapses. As a consequence, neurons can be classified into two main types: **excitatory neurons**, which increase the activity of their targets, and **inhibitory neurons**, which decrease it. The balance between excitation and inhibition is a central ingredient in shaping network activity.

At this point, a new modeling question arises: how should we describe mathematically the activity of neurons in a network?

The answer depends on the level of detail we want to retain. At one extreme, we can use networks of Hodgkin-Huxley neurons, which are very detailed but difficult to analyze and computationally expensive. A simpler alternative is to use networks of leaky integrate-and-fire neurons, which retain spike-based dynamics but are still challenging to study analytically. A further simplification leads to **firing rate models**, where the activity of a neuron is described not by individual spikes, but by its firing rate. These models are much more tractable and will be the main focus of what follows.

The motivation for firing rate models comes directly from experimental observations. Neural responses to stimuli are sequences of spikes, but these responses are noisy, with statistics often resembling a Poisson process and variability quantified by quantities such as the Fano factor. Because of this variability, it is often useful to consider the average response across trials, which is captured by the firing rate. Firing rate models therefore describe neural activity at a general level, neglecting precise spike timing and focusing on average behavior over timescales longer than that of individual spikes.

Mathematically, firing rate models describe how the activity of a neuron (or a population) evolves in time as a function of its inputs. The key idea is to separate two levels of description: the *input integration* and the *output response*.

Each neuron receives inputs from many presynaptic neurons: denoting by $u_i(t)$ the activity of the i -th presynaptic neuron and by w_i the corresponding synaptic weight, the total input is given by the weighted sum $\sum_i w_i u_i(t)$. This quantity represents the *total input* coming from the network. However, synaptic transmission is not instantaneous: inputs are first integrated into a synaptic current $I_s(t)$ (from my personal intuition, some kind of memory), which evolves with its own dynamics.

The entire process in a neuron is described by

$$\begin{cases} \tau_s \frac{dI_s(t)}{dt} = -I_s(t) + \sum_i w_i u_i(t), \\ \tau_r \frac{dv(t)}{dt} = -v(t) + F(I_s(t)). \end{cases}$$

The first equation describes how the synaptic current (memory) builds up and decays over time with characteristic timescale τ_s : each input contributes to I_s , but its effect is not permanent and decays exponentially. The second equation describes how the firing rate $v(t)$ evolves with its own timescale τ_r , relaxing toward a value determined by the current through the nonlinear function F .

Extremely dumb explanation for equation two made by me: now the neuron has a current

$I_s(t)$ and this current determines how much the neuron fires. But again, this is not instantaneous as the neuron does not jump immediately to the correct firing rate. Instead, $F(I_s(t))$ is the desired firing rate given the input, while $v(t)$ is the actual firing rate. The equation therefore says that the firing rate relaxes toward $F(I_s(t))$. The neuron is always firing at some rate, $v(t)$ just tells you how frequently and we want this frequently to be the one expressed by $F(I_s(t))$.

The introduction of these two variables, $I_s(t)$ and $v(t)$, reflects two distinct physical processes. The synaptic current I_s represents how inputs are filtered and accumulated at the membrane level, while the firing rate v represents the output activity of the neuron. In other words, I_s plays the role of an effective input current, and v is the observable output of the system.

Depending on the relative speed of these two processes, the equations can be simplified. If synaptic dynamics is much slower than rate dynamics ($\tau_s \gg \tau_r$), the firing rate adapts almost instantaneously to the current, and we can approximate

$$\begin{cases} \tau_s \frac{dI_s(t)}{dt} = -I_s(t) + \sum_i w_i u_i(t), \\ v(t) = F(I_s(t)). \end{cases}$$

In this regime, the current evolves slowly, while the firing rate simply follows it.

Conversely, if synaptic dynamics is much faster than rate dynamics ($\tau_s \ll \tau_r$), the current tracks the input almost instantaneously, and we can write

$$\begin{cases} I_s(t) = \sum_i w_i u_i(t), \\ \tau_r \frac{dv(t)}{dt} = -v(t) + F(I_s(t)). \end{cases}$$

In this case, the input is effectively instantaneous, and the slow variable is the firing rate itself.

The function F plays a central role in linking input and output. It represents how the neuron transforms input current into firing rate, and can be interpreted as a rough version of the f - I curve derived earlier for spiking neuron models. Common choices include a threshold-linear function

$$F(x) = [x - T]_+,$$

and a sigmoidal function

$$F(x) = \frac{1}{1 + \exp(-\beta(x - T))}.$$

More generally, F is chosen to match the average response of a neuron, neglecting precise spike timing but capturing how firing rate depends on input.

Finally, firing rate models are often used not to describe single neurons, but entire populations. In many brain areas, nearby neurons share similar selectivity properties and are more strongly connected. This allows us to group together neurons with similar charac-

teristics, such as excitatory or inhibitory neurons, or neurons tuned to similar stimuli, and describe them with a single average firing rate variable. In this way, complex networks of many neurons can be reduced to simpler systems that capture their collective behavior.

4.2 Feedforward and linear recurrent networks

In the previous section we introduced networks at a general level and motivated the use of firing-rate models as simplified descriptions of neural activity. We now make that discussion concrete: the main question is how different patterns of connectivity shape the transformation of activity through a network.

At this stage, the focus is not yet on complicated nonlinear dynamics, but on the two most important architectural ideas: **feedforward** connectivity, in which activity propagates from one layer to the next without lateral feedback, and **recurrent** connectivity, in which neurons within a layer also influence one another.

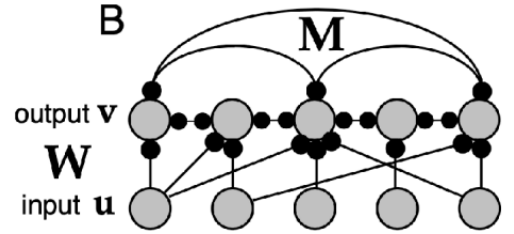
The conceptual shift is the following: in the single-neuron chapter, the problem was how a neuron transforms current into firing. Here, the problem is how a network transforms one pattern of activity into another.

The starting point is one of the rate-model formalism simplifications introduced in the previous deck: under the assumption of fast synaptic interactions, $\tau_s \ll \tau_r$, the activity variable satisfies

$$\tau_r \frac{dv(t)}{dt} = -v(t) + F(I_s(t)), \quad I_s(t) = \sum_i W_i u_i(t).$$

This leads to the basic two-layer model: if u denotes the activity of an input layer and v the activity of an output layer, then, assuming instantaneous synapses, the output dynamics is written as

$$\tau_r \frac{dv(t)}{dt} = -v(t) + F(Wu + Mv),$$



where W is the feedforward connectivity matrix from the input layer to the output layer, and M is the recurrent connectivity matrix inside the output layer.

This equation is important because it contains feedforward and recurrent networks as two limiting cases of the same general framework:

If $M = 0$, then neurons in the output layer do not interact laterally, and the model is purely feedforward.

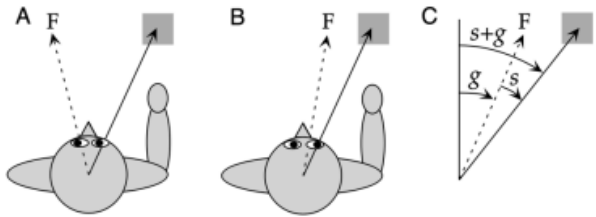
If $M \neq 0$, then output neurons also affect one another, and the model is recurrent. If recurrent interactions are much weaker than feedforward ones, then even a recurrent network may be approximated by a feedforward model. So feedforward and recurrent are not just names of two disconnected classes, they are two regimes within the same general equation.

Feedforward and recurrent architectures are not just abstract models, but correspond to different ways in which activity can be organized in a network. Some systems are well

approximated by feedforward structures, meaning that activity propagates layer by layer without strong internal interactions, while others are dominated by recurrent connections, where neurons continuously influence each other. The important point is not the biological detail, but the computational consequence: feedforward architectures mainly transform inputs into new representations, while recurrent architectures shape how activity evolves in time through amplification, suppression, or memory effects.

The slides then introduce concrete examples of **feedforward computations**, and the key idea in all of them is always the same: a neuron receives many inputs and simply adds them together with weights. The only question is what those inputs represent and what kind of pattern the neuron ends up detecting.

The first example is a **coordinate transformation problem**. Imagine you are looking at an object and want to reach it. The position of the object is first measured on your retina, so it depends on where the image falls in your eye, we call this quantity s i.e. the stimulus angle in retinal coordinates. However, your eyes can move, so the same object can produce a different value of s depending on where you are looking, so the position of your eyes is described by another variable, g , the gaze angle in body coordinates.

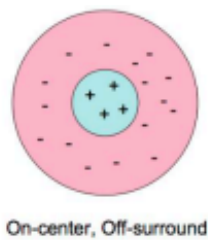


What we actually care about is the real position of the object relative to the body, which depends on both s and g . The important point is that neurons upstream already respond to both variables at the same time, so their activity can be written as $u_i(s, g)$: each neuron gives some signal that depends on both the stimulus and the gaze.

A downstream neuron then simply computes a weighted sum of these inputs,

$$v \sim \sum_i w_i u_i(s, g),$$

and fires if this sum is large enough. By choosing the weights w_i appropriately, the neuron becomes selective to the correct combination of s and g , effectively representing the real position of the object. The key idea is very simple: the neuron is not explicitly computing a formula like $s + g$, it is just combining inputs that already contain mixed information until the right pattern emerges.



The second example is **selectivity in the visual system (LGN)**: here the inputs u_i are even simpler as they represent small pieces of the visual scene, like whether light is present in a small region. At the earliest stage, neurons respond to very basic patterns, for example whether the center of a region is brighter than its surroundings. You can think of these as simple numbers u_i describing local contrast.

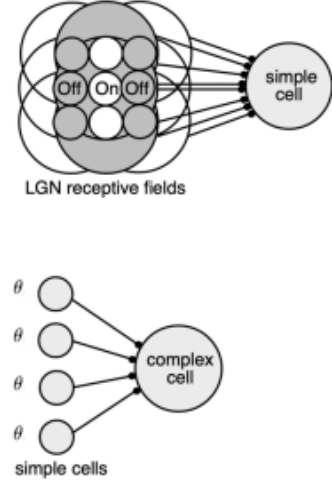
A downstream neuron again computes a weighted sum,

$$v = \sum_i w_i u_i,$$

and fires only if the inputs match a specific pattern. For instance, if the weights are arranged so that inputs along a line are added together, the neuron will fire when a line or edge is present in that orientation. If the pattern is not aligned, the sum remains small and the neuron does not respond.

In the **Hubel-Wiesel model (V1)** this idea is made explicit. Simple cells combine inputs arranged in space and therefore respond to a specific orientation and position of a bar of light. Complex cells then combine several simple cells that all detect the same orientation but at slightly different positions. This removes sensitivity to exact position while keeping sensitivity to orientation. In both cases, the computation is still just

$$v = F\left(\sum_i w_i u_i\right),$$



but by changing the weights, the neuron changes what pattern it detects. The general principle is that feedforward networks build more complex features step by step, simply by combining simpler inputs.

We then pass to **recurrent models**: the simplest recurrent system discussed in the slides is one-dimensional and can be interpreted either as a single self-interacting unit or as a homogeneous population described by one average activity variable v . The dynamics is

$$\tau \frac{dv}{dt} = -v + (h + Mv),$$

where h is the external input and M is now a scalar recurrent coupling. This equation can be understood very directly: the term $-v$ means that, if nothing happens, the activity naturally decays back to zero; the term h is the external drive, pushing the activity up; the new term Mv means that the neuron (or population) feeds its own activity back into itself.

If $M > 0$, the neuron reinforces its own activity (excitatory feedback); if $M < 0$, it suppresses it (inhibitory feedback). So the key idea of recurrence is very simple: the current state of the system influences its future evolution.

The slides provide the exact solution:

$$v(t) = \frac{h}{1 - M} \left[1 - \exp\left(-\frac{(1 - M)t}{\tau}\right) \right] + v(0) \exp\left(-\frac{(1 - M)t}{\tau}\right).$$

To understand this, focus on the exponential term: its decay rate is $(1 - M)/\tau$, so everything depends on the sign of $1 - M$.

1. If $M > 1$, then $1 - M < 0$ and the exponential grows: activity increases without bound. This is called instability, and it simply means that small inputs get amplified uncontrollably.
2. If instead $M < 1$, then $1 - M > 0$ and the exponential decays: the system forgets its initial condition and converges to a stable value. This is what “stable” means here: activity stays bounded and settles to a finite level. In such case, the system converges to the fixed point

$$v_\infty = \frac{h}{1 - M}.$$

This expression is very intuitive:

- If $0 < M < 1$, the feedback is excitatory, so the neuron boosts its own activity. As a result, the output v_∞ is larger than the input h : this is amplification. At the same time, the effective timescale becomes $\frac{\tau}{1-M}$, which is larger than τ , meaning that the system reacts more slowly. Intuitively, this happens because activity is being re-injected into the system, so it takes longer to settle.
 - If instead $M < 0$, the feedback is inhibitory: the neuron suppresses its own activity. The output is smaller than the input (attenuation), and the timescale becomes shorter, meaning the system reacts more quickly. In this case, activity is actively damped.
3. A special case occurs when $M = 1$ as here, decay and feedback exactly cancel. The system no longer relaxes to a fixed value, but instead integrates the input:

$$v(t) = \frac{1}{\tau} \int^t h(t') dt'.$$

This is called a perfect integrator, and it means that the neuron accumulates input over time. Intuitively, instead of forgetting past input, the system keeps adding it up.

Linear recurrent networks - N dimensional: we now generalize this idea to N dimensions. The dynamics becomes

$$\tau_r \frac{dv}{dt} = -v + h + Mv,$$

where v and h are now vectors and M is a matrix describing connections between neurons. The key idea is that in higher dimensions, activity is not just a number but a pattern, and recurrence can affect different patterns in different ways.

To analyze this, we decompose the activity along the eigenvectors e_ν of M :

$$v(t) = \sum_{\nu=1}^{N_v} C_\nu(t) e_\nu.$$

Each eigenvector represents a particular pattern of activity across neurons. The important point is that each pattern evolves independently, and its behavior is controlled by the corresponding eigenvalue λ_ν . If M is symmetric, the eigenvalues are real and the interpretation is simple.

1. If $\lambda_\nu > 1$, that pattern is unstable and grows over time. So stability or instability is

now not a global property, but depends on the direction in activity space.

2. If $\lambda_\nu < 1$, the corresponding pattern is stable and the steady state is

$$v_\infty = \sum_\nu \frac{e_\nu h}{1 - \lambda_\nu} e_\nu.$$

This formula shows that the network does not treat all inputs equally. Each component of the input along direction e_ν is amplified by the factor $1/(1 - \lambda_\nu)$. If λ_ν is close to 1, that component is strongly amplified. This is what the slides mean by “selective amplification”: the network amplifies some patterns more than others.

3. Finally, if one eigenvalue satisfies $\lambda_1 = 1$, the network integrates the input along that specific direction, just like the one-dimensional case. This means that the network can store information, but only for certain patterns.

The same idea extends to **continuous networks**: instead of discrete neurons, activity depends on a continuous variable θ , and connectivity is described by a kernel

$$M(\theta - \theta') = \frac{\lambda_1}{\pi} \cos(\theta - \theta').$$

In this case, activity can be decomposed into Fourier modes. The network selectively amplifies the mode $\mu = 1$, while leaving others weaker. This is the continuous analogue of eigenvector amplification: recurrence picks out specific patterns and enhances them.

In summary, recurrence does not just change the overall strength of activity. It determines which patterns are amplified, which are suppressed, and which can be maintained over time. This is the key difference with feedforward networks, which only combine inputs, while recurrent networks actively shape the dynamics of activity.